

Data Wrangling

Shu Guo

2025-03-24

YAML

Introduction

The tidyverse is a collection of R packages that share a high-level design philosophy and low-level grammar and data structures, so that learning one package makes it easier to learn the next. The package provides a very powerful way of manipulating data, and we can use this package to accomplish our work easily. While here I will mostly consider using functions from tidyverse package, I will re-visit functions from base R or other packages to achieve the same goals. The tidyverse package can be installed by using a single R command:

```
install.packages("tidyverse")
```

Key packages in the tidyverse include:

- ggplot2: For data visualization.
- dplyr: for data manipulation.
- tidyr: for data tidying.
- readr: for data import.
- purrr: for functional programming.
- tibble: for tibbles, a modern re-imagining of data frames.
- stringr: for strings.
- forcats for factors.
- lubridate: for date/times.
- haven: for reading SAS datasets into R.

When we run `library` command, R will load and attach the four core packages in tidyverse, which include `ggplot`, `tibble`, `tidyr` and `readr`.

```
library("tidyverse")
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.3      v tibble     3.3.1
```

```
## v lubridate 1.9.5      v tidyr      1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()      masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
library("nycflights13")
```

The conflicts message here means the two functions, `filter` and `lag`, from `tidyverse` package are now masking the two functions with the same names from R base package `stats`. One of the important features in `tidyverse` is pipe, `%>%` from package `magrittr` or `|>` from R base. R Pipe starts with a dataset, and will pass the object at its left to the function on its right side and generate results:

```
diamonds |>                # start with diamonds dataset
  filter(cut == 'Good')    # to select records with cut = 'Good'
```

```
## # A tibble: 4,906 x 10
##   carat cut    color clarity depth table price     x     y     z
##   <dbl> <ord> <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Good  E     VS1     56.9    65   327  4.05  4.07  2.31
## 2  0.31 Good  J     SI2     63.3    58   335  4.34  4.35  2.75
## 3  0.3   Good  J     SI1     64     55   339  4.25  4.28  2.73
## 4  0.3   Good  J     SI1     63.4    54   351  4.23  4.29  2.7
## 5  0.3   Good  J     SI1     63.8    56   351  4.23  4.26  2.71
## 6  0.3   Good  I     SI2     63.3    56   351  4.26  4.3   2.71
## 7  0.23 Good  F     VS1     58.2    59   402  4.06  4.08  2.37
## 8  0.23 Good  E     VS1     64.1    59   402  3.83  3.85  2.46
## 9  0.31 Good  H     SI1     64     54   402  4.29  4.31  2.75
## 10 0.26 Good  D     VS2     65.2    56   403  3.99  4.02  2.61
## # i 4,896 more rows
```

Subsetting Rows and Columns in a Dataset

Rows

The `filter` function can be used to select records from a dataset that meet the conditions specified in its arguments.

```
# select records with price greater than 10000
diamonds |>
  filter(price > 10000) |>
  print(n = 5)
```

```
## # A tibble: 5,222 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  1.7 Ideal    J      VS2     60.5  58 10002  7.73  7.74  4.68
## 2  1.03 Ideal    E      VVS2    60.6  59 10003  6.5   6.53  3.95
## 3  1.23 Very Good G      VVS2    60.6  55 10004  6.93  7.02  4.23
## 4  1.25 Ideal    F      VS2     61.6  55 10006  6.93  6.96  4.28
## 5  2.01 Very Good I      SI2     61.4  63 10009  8.19  7.96  4.96
## # i 5,217 more rows
```

```
# This is the same as the following R code to selecting row in base R
head(diamonds[diamonds$price > 10000, ], 5)
```

```
## # A tibble: 5 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  1.7 Ideal    J      VS2     60.5  58 10002  7.73  7.74  4.68
## 2  1.03 Ideal    E      VVS2    60.6  59 10003  6.5   6.53  3.95
## 3  1.23 Very Good G      VVS2    60.6  55 10004  6.93  7.02  4.23
## 4  1.25 Ideal    F      VS2     61.6  55 10006  6.93  6.96  4.28
## 5  2.01 Very Good I      SI2     61.4  63 10009  8.19  7.96  4.96
```

```
# select records with ideal cut and best color
diamonds |>
  filter(cut == "Ideal" & color == "D") |>
  print(n = 5)
```

```
## # A tibble: 2,834 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.3 Ideal    D      SI1     62.5  57  552  4.29  4.32  2.69
## 2  0.3 Ideal    D      SI1     62.1  56  552  4.3   4.33  2.68
## 3  0.71 Ideal    D      SI2     62.3  56 2762  5.73  5.69  3.56
## 4  0.71 Ideal    D      SI1     61.9  59 2764  5.69  5.72  3.53
## 5  0.71 Ideal    D      SI2     61.6  55 2767  5.74  5.76  3.54
## # i 2,829 more rows
```

```
# same base R code as below
head(diamonds[diamonds$cut == "Ideal" & diamonds$color == "D", ], 5)
```

```
## # A tibble: 5 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.3 Ideal    D      SI1     62.5  57  552  4.29  4.32  2.69
## 2  0.3 Ideal    D      SI1     62.1  56  552  4.3   4.33  2.68
## 3  0.71 Ideal    D      SI2     62.3  56 2762  5.73  5.69  3.56
## 4  0.71 Ideal    D      SI1     61.9  59 2764  5.69  5.72  3.53
## 5  0.71 Ideal    D      SI2     61.6  55 2767  5.74  5.76  3.54
```

arrange

The `arrange` function will sort the dataset based on values from selected columns in ascending order. To sort in descending order, we can use the `desc()` function. The following R code first sorts the dataset by the first three columns in ascending order and then sorts it by the fourth column in descending order:

```
diamonds |>
  arrange(color, desc(carat)) |> # sort by color and carat in descending order
  print(n = 5)
```

```
## # A tibble: 53,940 x 10
##   carat cut          color clarity depth table price      x      y      z
##   <dbl> <ord>        <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  3.4 Fair         D      I1     66.8   52 15964  9.42  9.34  6.27
## 2  2.75 Ideal       D      I1     60.9   57 13156  9.04  8.98  5.49
## 3  2.58 Very Good D      SI2     58.9   63 14749  9.08  9.01  5.33
## 4  2.57 Premium    D      SI2     58.9   58 17924  8.99  8.94  5.28
## 5  2.26 Very Good D      SI2     61.6   61 18028  8.47  8.38  5.19
## # i 53,935 more rows
```

```
# same base R code as below
head(diamonds[order(diamonds$color, -diamonds$carat), ], 5)
```

```
## # A tibble: 5 x 10
##   carat cut          color clarity depth table price      x      y      z
##   <dbl> <ord>        <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  3.4 Fair         D      I1     66.8   52 15964  9.42  9.34  6.27
## 2  2.75 Ideal       D      I1     60.9   57 13156  9.04  8.98  5.49
## 3  2.58 Very Good D      SI2     58.9   63 14749  9.08  9.01  5.33
## 4  2.57 Premium    D      SI2     58.9   58 17924  8.99  8.94  5.28
## 5  2.26 Very Good D      SI2     61.6   61 18028  8.47  8.38  5.19
```

select

We use `'select()'` to work on columns, or variables, and we can select any columns using variable names in a dataset:

```
starwars %>% # a dataset with 14 variables
  head(n = 3)
```

```
## # A tibble: 3 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender
##   <chr>    <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>
## 1 Luke Sky~   172    77 blond      fair       blue          19 male  mascu~
```

```
## 2 C-3PO          167    75 <NA>         gold        yellow          112 none  mascu~
## 3 R2-D2           96    32 <NA>         white, bl~ red          33 none  mascu~
## # i 5 more variables: homeworld <chr>, species <chr>, films <list>,
## #   vehicles <list>, starships <list>
```

```
starwars %>%
  select(homeworld, height, mass) %>% # select three variables from starwars datasets
  print(n = 3)
```

```
## # A tibble: 87 x 3
##   homeworld height  mass
##   <chr>      <int> <dbl>
## 1 Tatooine    172    77
## 2 Tatooine    167    75
## 3 Naboo       96    32
## # i 84 more rows
```

```
starwars %>%
  select(eye_color:films) %>% # select a range of variables using colon
  print(n = 3)
```

```
## # A tibble: 87 x 7
##   eye_color birth_year sex   gender   homeworld species films
##   <chr>      <dbl> <chr> <chr>    <chr>    <chr> <list>
## 1 blue           19 male  masculine Tatooine Human  <chr [5]>
## 2 yellow         112 none  masculine Tatooine Droid  <chr [6]>
## 3 red            33 none  masculine Naboo    Droid  <chr [7]>
## # i 84 more rows
```

```
starwars %>%
  select(starts_with("s") | ends_with("color")) %>% # select variables start with "s"
  print(n = 3) # or end with "color"
```

```
## # A tibble: 87 x 6
##   skin_color sex   species starships hair_color eye_color
##   <chr>      <chr> <chr>    <list>    <chr>    <chr>
## 1 fair      male  Human  <chr [2]> blond    blue
## 2 gold      none  Droid  <chr [0]> <NA>     yellow
## 3 white, blue none  Droid  <chr [0]> <NA>     red
## # i 84 more rows
```